



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title
CS4051NI Fundamental of Computing

70% Individual Coursework
Submission : Final Submission

Academic Semester: Summer Semester 2025

Credit: 15 credit semester long module

Student Name: Raunit Giri

London Met ID: 24058860

College ID: NP01CP4S250011

Assignment Due Date: Wednesday, August 27, 2025

Assignment Submission Date: Wednesday, August 27, 2025

Submitted To:

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

C:\Users\Raunit\OneDrive\Desktop\Introduction.docx

 Islington College, Nepal

Document Details

Submission ID

trn:oid::3618:109784655

Submission Date

Aug 27, 2025, 12:36 PM GMT+5:45

Download Date

Aug 27, 2025, 12:39 PM GMT+5:45

File Name

C:\Users\Raunit\OneDrive\Desktop\Introduction.docx

File Size

37.9 KB

25 Pages

4,916 Words

28,594 Characters

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **50 Not Cited or Quoted 10%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **11 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 11%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 50 Not Cited or Quoted** 10%
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations** 0%
Matches that are still very similar to source material
- 11 Missing Citation** 2%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 1% Publications
- 11% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Sheffield Hallam University on 2023-04-15	1%
2	Submitted works	Sheffield Hallam University on 2023-05-20	<1%
3	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-12-05	<1%
4	Submitted works	University of Western Sydney on 2015-05-25	<1%
5	Internet	community.esri.com	<1%
6	Submitted works	Montana State University, Bozeman on 2015-01-27	<1%
7	Internet	ijrpr.com	<1%
8	Submitted works	Softwarica College of IT & E-Commerce on 2013-07-30	<1%
9	Submitted works	Kaplan International Colleges on 2025-05-28	<1%
10	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-03-14	<1%

11	Submitted works	Oxford Brookes University on 2024-12-06	<1%
12	Submitted works	CTI Education Group on 2025-06-28	<1%
13	Submitted works	QA Learning on 2024-03-22	<1%
14	Submitted works	Sagar Lohani on 2025-06-11	<1%
15	Submitted works	American University in the Emirates on 2024-06-26	<1%
16	Submitted works	University of South Florida on 2015-04-14	<1%
17	Submitted works	Info Myanmar College on 2024-10-28	<1%
18	Submitted works	National University of Technology (NUTECH)-Islamabad on 2025-08-06	<1%
19	Submitted works	University of Wales Institute, Cardiff on 2022-04-08	<1%
20	Internet	www.slideshare.net	<1%
21	Submitted works	Wayne State University on 2024-10-19	<1%
22	Submitted works	St Edmund's on 2013-06-05	<1%
23	Submitted works	University of Huddersfield on 2025-05-09	<1%
24	Internet	mt940.readthedocs.io	<1%

25	Submitted works	RDI Distance Learning on 2025-07-16	<1%
26	Submitted works	University of Salford on 2024-04-15	<1%
27	Submitted works	University of Strathclyde on 2010-04-19	<1%
28	Internet	www.coursehero.com	<1%
29	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2023-02-13	<1%
30	Submitted works	University of Wolverhampton on 2025-05-18	<1%

Table of Contents

1	Table of Figures	1
2	Table of Tables	3
3	Introduction	4
3.1	Aim and Objective	4
3.1.1	Aim.....	4
3.1.2	Objective.....	5
3.2	Technologies Involved	5
3.2.1	Python.....	5
3.2.2	IDLE.....	6
3.2.3	Notepad	6
3.2.4	Ms. Word	7
3.2.5	Draw.io.....	7
4	Algorithm.....	8
5	Flow Chart.....	10
6	Pseudocodes	14
6.1	Main.py	14
6.2	Read.py.....	18
6.3	Write.py.....	18
6.4	Display.py	19
6.5	Generate_bill.py.....	20
6.6	Operations.py.....	24
7	Data Structures	32
7.1	Primitive data	32
7.1.1	String	32
7.1.2	Integer.....	33
7.1.3	Float.....	34
7.1.4	Boolean.....	34
7.2	Complex Data Structures	35
7.2.1	Tuples	35

7.2.2	List	36
7.2.3	Dictionary	37
7.2.4	Set	38
8	Execution of Porgram.....	39
8.1	Display Inventory	39
8.2	Stock product.....	39
8.2.1	Add new Product	40
8.2.2	Restock product.....	41
8.3	Sell product.....	42
8.3.1	Add product to cart	43
8.3.2	Show Cart.....	43
8.3.3	Buy and Generate Bill.....	44
8.4	EXIT.....	44
9	Testing	45
9.1	Test 1	45
9.2	Test 2.....	46
9.3	Test 3.....	47
9.4	Test 4.....	50
9.5	Test 5.....	52
10	Conclusion	53
11	Appendix.....	53

1 Table of Figures

Figure 2-1 Python.....	5
Figure 2-2 IDLE (Integrated Development and Learning Environment).....	6
Figure 2-3 Notepad	6
Figure 2-4 Microsoft Word	7
Figure 2-5 Draw.io.....	7
Figure 4-1 Flow Chart Page 1	10
Figure 4-2 Flow Chart Page 2	11
Figure 4-3 Flow Chart Page 3	11
Figure 4-4 Flow Chart Page 4	12
Figure 4-5 Flow Chart Page 5	13
Figure 6-1 String Usage	33
Figure 6-2 Integer usage	33
Figure 6-3 Usage of float.....	34
Figure 6-4 Boolean usage	35
Figure 6-5 Tuple usage	36
Figure 6-6 Usage of list	36
Figure 6-7 Usage of dictionary	37
Figure 6-8 Usage of set.....	38
Figure 7-1 Display Inventory	39
Figure 7-2 Stock Product.....	40
Figure 7-3 Add new Product.....	40
Figure 7-4 Invoice for Add new Product	41
Figure 7-5 Restock Product.....	41
Figure 7-6 Invoice for Restock Product	42
Figure 7-7 Sell Product.....	42
Figure 7-8 Add product to cart.....	43
Figure 7-9 Display Cart	43
Figure 7-10 Buy and Generate Bill	44
Figure 7-11 Invoice for Customer	44
Figure 7-12 Exit.....	44
Figure 8-1 Test 1 try, catch input field	45
Figure 8-2 Test 1 try, catch helper function	45
Figure 8-3 Test 1 try, catch Output.....	45
Figure 8-4 Test 2 Negative input : input field code	46
Figure 8-5 Test 2 Negative input : value check	46
Figure 8-6 Test 2 Negative input: Output	46
Figure 8-7 Test 2 Non- exist value : input field code	47
Figure 9-8 Test 2 Non-exist value : value check	47
Figure 8-9 Non-exist: Output	47
Figure 8-10 Test 3: Stock Operation menu	48
Figure 8-11 Test 3 Restock 2 products	48
Figure 8-12 Test 3 Restock product Invoice.....	49

Figure 8-13 Test 4 Customer menu	50
Figure 8-14 Test 4 Adding product to cart (1).....	50
Figure 8-15 Test 4 Adding product to cart (2).....	51
Figure 8-16 Test 4 Buy and Generate bill.....	51
Figure 8-17 Test 4 Customer invoice	51
Figure 8-18 Test 5: Inventory before Test 3	52
Figure 8-19 Test 5: Inventory after Test 3	52
Figure 8-20 Test 5: Inventory before Test 4	52
Figure 8-21 Test 5: Inventory after Test 4	52

2 Table of Tables

Table 1 Test 1 Table	45
Table 2 Test 2 Table	46
Table 3 Test 3 Table	47
Table 4 Test 4 Table	50
Table 5 Test 5 Table	52

3 Introduction

This project is to develop a Shoes Wholesale System using Python as the programming language for a private wholesaler that sales shoe of different brands, both international and domestic. This developed program is able to read a text file where data such as shoe name, brand, no of items available, price and source are to be found in a formatted text and display all the available shoes. It is also able to generate an invoice (.txt) for each transaction and keep the record of transaction stored in another .txt file. As a Wholesale System, when a customer buys in quantity more then 10 during a transaction, there is a 5% discount on that transaction. It provides extra discount i.e. 7% discount for more then 10 items if the product is sourced domestic. The availability of the shoes is updated with every transaction. The generated invoice contains shoe. Brand, quantity sold, name of customer, date, discount, and total amount. An invoice is also generated when the System owner restocks shoes from different vendors. This contains the name of supplier, shoe name, rate of shoes, quantity bought, brand, date and total amount. After restock invoice is created, items are updated on the main file. The format for each invoice can be changed as the project progresses as per the requirement of the system.

3.1 Aim and Objective

3.1.1 Aim

The aim of this project is to be able to implement python in a real-world problem scenario. Using Python to design and implement this Shoe Wholesale System that is able to store, manage and display product data for a private wholesale shoe company. It demonstrates file handling capabilities of python by using operations to read and write structured data into plain text file i.e. .txt file and to show how data structures are used to store and manipulate data.

3.1.2 Objective

The objectives of this project are:

- 1) Develop a Python program that can read, write and process data stored in external file (.txt)
- 2) To show the processed data in a neatly formatted format in IDLE Shell window
- 3) Update the data in the external stored data after certain events like after a transaction or restock
- 4) Create new .txt file after each transaction with the customer name as file name
- 5) Prepare a project report using Microsoft Word that documents the data structure, algorithm and more

3.2 Technologies Involved

3.2.1 Python

Python is the programming language used for this project. Python is a high level, interpreted programming language that was introduced in 1991. It started to become more popular since 2010 because of its flexibility and ease to learn & use. It is used in web development, data science, automation, AI and more. Since it is backed by library support, programmers don't have to build everything from scratch and find/ use the library that does what a programmer needs. This project is purely python with no support from other languages and was code on Python version 3.13.4.

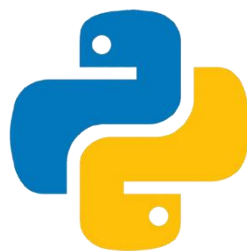


Figure 2-1 Python

3.2.2 IDLE

IDLE (Integrated Development and Learning Environment) is a development environment that is installed with python. IDLE can be used to execute a single statement but also to create, modify and execute python files. IDLE provides fully-featured text editor to create python script with additional features to help programmers. Since it is most suitable for small Python projects, we are using this to create file in Editor window and execute the python file in Shell window of IDLE.



Figure 2-2 IDLE (Integrated Development and Learning Environment)

3.2.3 Notepad

Notepad is the basic text editor that is available in every windows OS. It is used to create and edit plain text document i.e. file with .txt extension. It comes with the most basic Text formatting functions and has a simple window design. It is perfect to create and view .txt files with its simple design. In this project catalogue.txt file is manually created with given data whereas other .txt files are created by Python script. Notepad is used to view the resultant file and verify data in catalogue.txt.



Figure 2-3 Notepad

3.2.4 Ms. Word

Microsoft Word is a word process program designed by Microsoft. Unlike Notepad Ms. Word is used for creating text files that are rich in text and formatting. It is also included in Windows OS. Ms. Word has become the default/most common word processing program in the world. Here, this software is used to create an academic report about the project which includes introduction to project, aim & Objective, Technologies used and Data structures. The made document is then exported in PDF for easier submission.



Figure 2-4 Microsoft Word

3.2.5 Draw.io

Draw.io is a software / online diagramming tool that helps making diagrams and charts. This software has a wide selection of shapes and visual elements that help speed up the process of making charts. This tool features a drag and drop interface which makes it user friendly. Since this is an online tool, it can be accessed by any device i.e. cross platform access, Cloud integration with services like Google Drive, OnDrive and GitHub. It also allows real-time collaboration with others online. In this project Draw.io was used to create and manage flowchart.



Figure 2-5 Draw.io

4 Algorithm

The step-by-step set of instructions made to solve or accomplish a task is called Algorithm. The Algorithm for this project goes as follows:

- Step 1 :** Choose Main Option
- Step 2 :** If option is 1 goto Step 3
if option is 2 goto Step 4
if option is 3 goto Step 40
if option is 4 goto Step 55
Else goto Step 1
- Step 3 :** Display Inventory table
- Step 4 :** Choose Stock Option
- Step 5 :** If option is 1 goto Step 6
if option is 2 goto Step 24
if option is 3 goto Step 1
Else goto Step 4
- Step 6 :** Enter Supplier Name
- Step 7 :** Enter product id,
- Step 8 :** If product id already exists goto step 7
If product id is "EXIT" goto Step 4
Else goto Step 9
- Step 9 :** Enter product Name
- Step 10 :** Enter units purchased
- Step 11 :** If units purchased is valid goto step 12
If units purchased is "EXIT" goto Step 4
Else got to step 10
- Step 12 :** Enter product Brand
- Step 13 :** Enter product Buying price
- Step 14 :** If units purchased is valid goto step 15
Else got to step 13
- Step 15 :** Enter product selling price
- Step 16 :** If units purchased is valid goto step 17
Else got to step 15
- Step 17 :** Enter Source
- Step 18 :** If source is International or Domestic goto Step 19
Else goto Step 17
- Step 19 :** Choose add another product option
- Step 20 :** If Option is n goto Step 21
Else goto Step 6
- Step 21 :** Update inventory
- Step 22 :** Create invoice
- Step 23 :** Display success message and Goto Step 4
- Step 24 :** Enter Supplier Name

Step 25 : Enter product id
Step 26 : If product id exists goto step 25
 If product id is "EXIT" goto Step 4
 Else goto Step 27
Step 27 : Display product data
Step 28 : Enter units purchased
Step 29 : If units purchased is valid goto step 30
 If product id is "EXIT" goto Step 4
 Else got to step 28
Step 30 : Enter purchase price
Step 31 : Choose Option
Step 32 : If option is y goto Step 33
 Else goto Step 34
Step 33 : Enter new selling price
Step 34 : If units purchased is valid goto step 35
 Else got to step 33
Step 35 : Choose add another product option
Step 36 : If Option is n goto Step 37
 Else goto Step 24
Step 37 : Update inventory
Step 38 : Create invoice
Step 39 : Display success message and Goto Step 4
Step 40 : Enter Customer name
Step 41 : Choose option
Step 42 : If option is 1 goto Step 43
 if option is 2 goto Step 44
 if option is 3 goto Step 51
 if option is 4 goto Step 1
 Else goto Step 40
Step 43 : Display customer cart
Step 44 : Enter product id
Step 45 : If product id exists goto step 46
 If product id is "EXIT" goto Step 41
 Else goto Step 44
Step 46 : Display product data
Step 47 : Enter units purchased
Step 48 : If units purchased is greater than available stock and is valid goto Step 47
 If units purchased is "EXIT" goto Step 41
 Else goto 49
Step 49 : Add product to cart
Step 50 : Display success message and Goto Step 41
Step 51 : If cart empty goto step
 Else goto Step 53

Step 52 : Update inventory data

Step 53 : Create invoice

Step 54 : Display Success message and goto Step 1

Step 55 : END

5 Flow Chart

The step-by-step diagram instruction made to solve or accomplish a task is called Flowchart. It is similar to algorithm but is a graphical approach The Flow Chart for this project goes as follows:

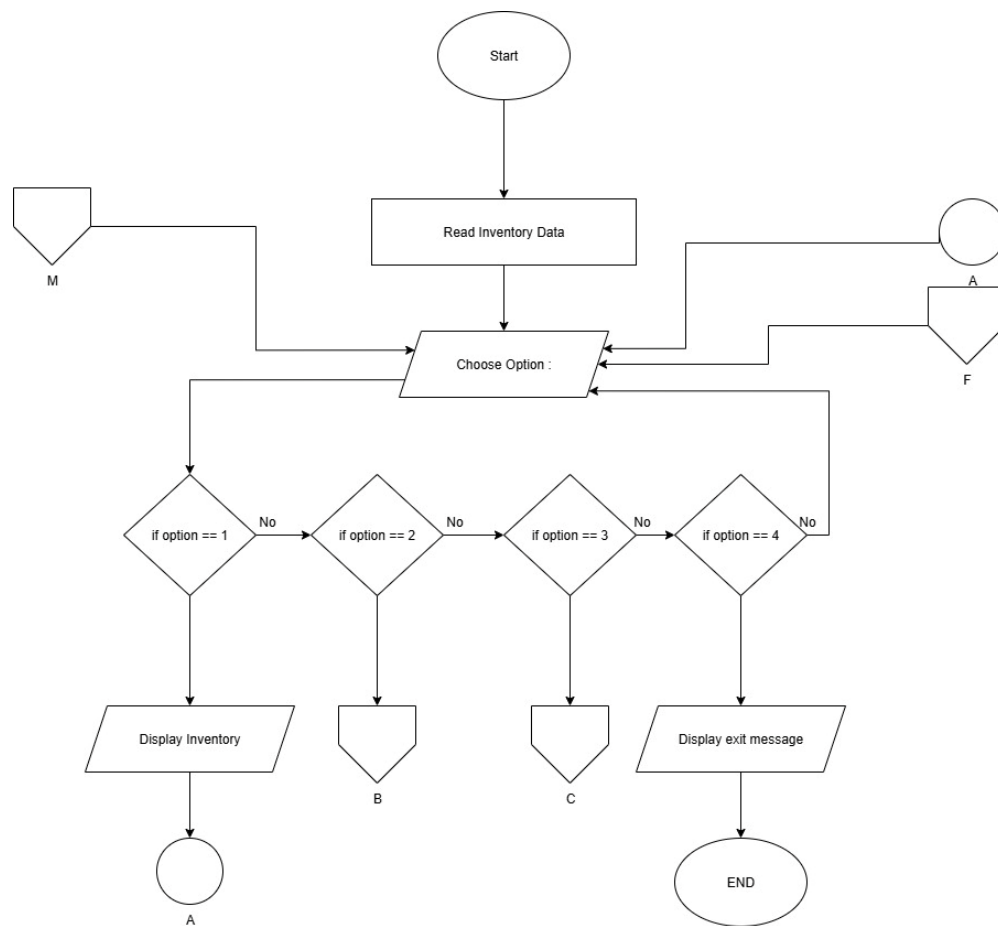


Figure 4-1 Flow Chart Page 1

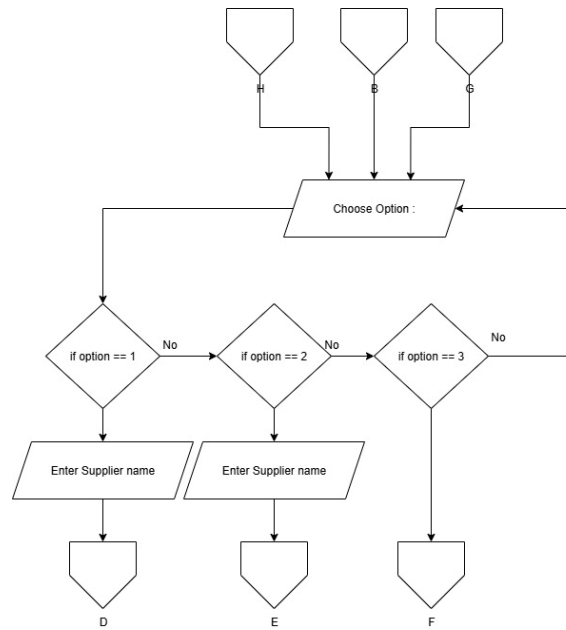


Figure 4-2 Flow Chart Page 2

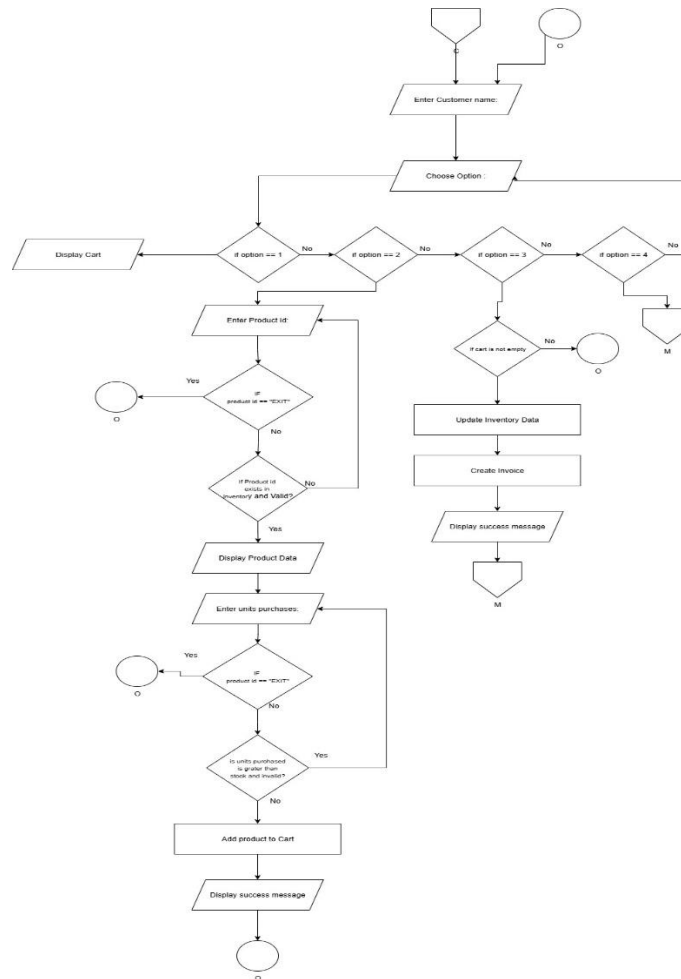


Figure 4-3 Flow Chart Page 3

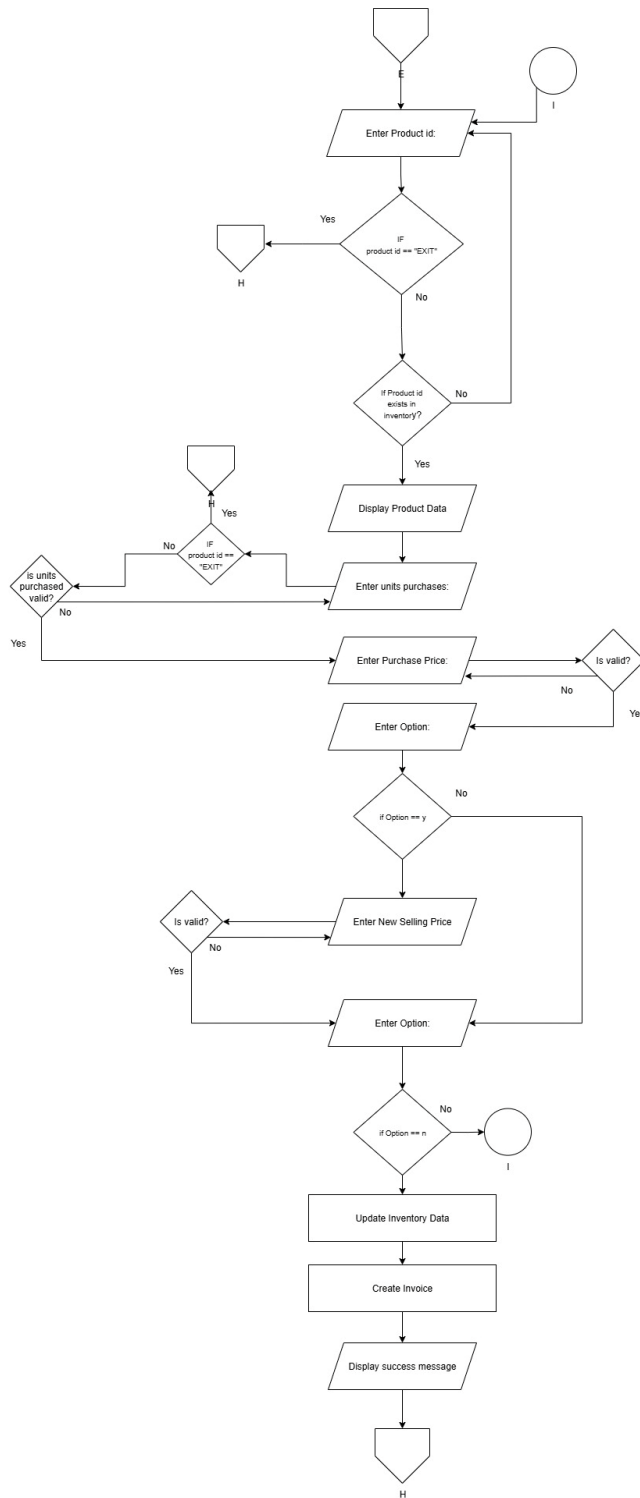


Figure 4-4 Flow Chart Page 4

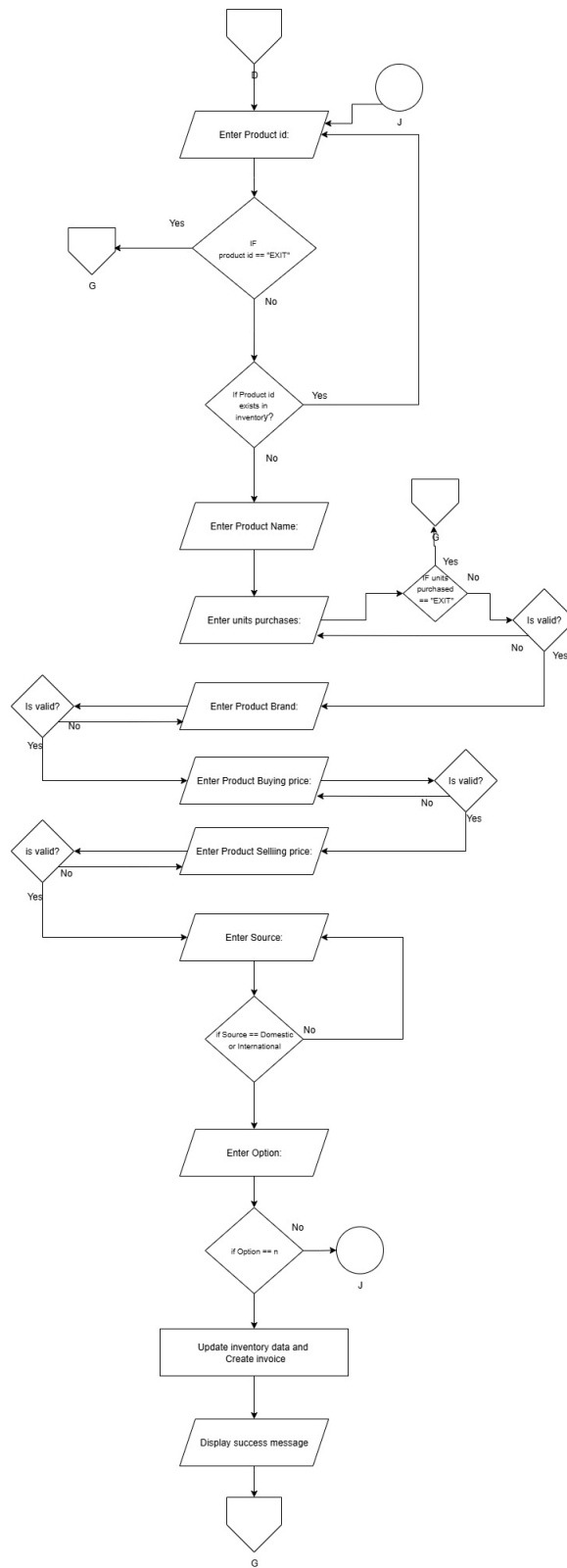


Figure 4-5 Flow Chart Page 5

6 Pseudocodes

6.1 Main.py

Import Read

Import Display

Import Operations

Import Write

from Generate_bill import generate_bill

Function customer_operation(inventory):

 boughtlist = []

 customername = input("Enter Customer Name : ")

While True:

OUTPUT("\n"+"***"25)

OUTPUT ("*{: ^48}*".format("Customer OPERATIONS"))

OUTPUT ("*"+" "48+"*")

OUTPUT ("* {:<44} *".format("Customer Name : "+customername))

OUTPUT ("*"+" "48+"*")

OUTPUT ("* {:<25} {:<20}*".format("1.Show Cart ", "2.Add product"))

OUTPUT ("*"+" "48+"*")

OUTPUT ("* {:<25} {:<20}*".format("3.Buy and Generate Bill",
"4.Cancel purchase"))

OUTPUT ("*"+" "48+"*")

OUTPUT ("***"25+"\n")

IF Selected == "1":

IF boughtlist:

 Display.display_buylist(boughtlist)

ELSE:

OUTPUT("n**Empty Cart**n")

END IF

ELIF selected == "2":

```

Display.display_available(inventory)
purchased_item = Operations.customer_sell(inventory)
IF purchased_item:
    boughtlist.append(purchased_item)
END IF
ELIF (selected == "3"):
    IF boughtlist:
        success = generate_bill(customername,boughtlist)
        IF success:
            Write.write_inventory(inventory)
        ELSE:
            OUTPUT("Purchase Cancelled")
        END IF
        Break
    ELSE:
        OUTPUT("Empty Cart")
    END IF
ELIF(selected == "4"):
    IF boughtlist:
        Operations.canceled_restock(inventory,boughtlist)
    END IF
        OUTPUT("Purchase Cancelled")
    ELSE:
        OUTPUT("Invalid Entry")
    END IF
END Function

Function stock_operation(inventory):

```

While True:

OUTPUT("\\n"+"*****25)

OUTPUT ("*{: ^48}*".format("STOCK OPERATIONS"))

OUTPUT ("*"+" " *48+"*")

OUTPUT ("* {:<25} {:<20}*".format("1.Add new product ",
"2.Restock product"))

OUTPUT ("*"+" " *48+"*")

OUTPUT ("* {:<25} {:<20}*".format("3.Back",""))

OUTPUT ("*"+" " *48+"*")

OUTPUT ("*****25+\\n\\n")

Selected = input("\\nSelect Operation : ")

IF Selected == "1":

 suppliername,boughtlist = Operations.add_product(inventory)

IF boughtlist:

 success = generate_bill(suppliername,boughtlist,False)

IF success:

 Write.write_inventory(inventory)

END IF

END IF

ELIF selected == "2":

 Display.display_inventory(inventory)

 suppliername,boughtlist = Operations.restock_product(inventory)

IF boughtlist:

 success = generate_bill(suppliername,boughtlist,False)

IF success:

 Write.write_inventory(inventory)

END IF

END IF


```

    ELIF (selected == "3"):
        OUTPUT("\\nBack to main menu\\n")
        break
    ELSE:
        OUTPUT("\\nInvalid Entry\\n")
END IF

```

END Function

```
inventory_data = Read.read_inventory()
```

While True:

```

    OUTPUT("\\n"+"***25")
    OUTPUT ("*{: ^48}*".format("OPERATIONS"))
    OUTPUT ("**"+" " *48+"**")
    OUTPUT ("* {:<25} {:<20}*".format("1.Display Inventory ", "2.Stock product"))
    OUTPUT ("**"+" " *48+"**")
    OUTPUT ("* {:<25} {:<20}*".format("3.Sell Product", "4.EXIT"))
    OUTPUT ("**"+" " *48+"**")
    OUTPUT ("***25+\\n")
    Selected = input("\\nSelect Operation : ")
    IF Selected == "1":
        Display.display_inventory(inventory_data)
    ELIF selected == "2":
        stock_operation(inventory_data)
    ELIF (selected == "3"):
        customer_operation(inventory_data)
    ELIF(selected == "4"):
        OUTPUT("\\n**EXIT PROGRAM**\\n")
        break

```

ELSE:

OUTPUT("Invalid Entry\n")

END IF

6.2 Read.py

Function read_inventory():

result_list=[]

Try:

With open("Catalogue.txt","r") as file:

lines = file.readlines()

FOR line in lines:

parts = line.replace("\n","").split(",")

parts[3]=int(parts[3])

parts[4]=float(parts[4])

result_list.append(parts)

END FOR

return result_list

Except FileNotFoundError:

OUTPUT("File not found\n")

Except Exception as e:

OUTPUT("Failed Operation : " + str(e))

END Try

END Function

6.3 Write.py

Function write_inventory(inventory):

Try:

With open("Catalogue.txt", "w") as file:

For row in inventory:

line = "{},{},{},{},{},{}\n".format(

row[0],row[1],row[2],row[3],row[4],row[5])

```
file.write(line)
```

```
END For
```

```
Except Exception as e:
```

```
    OUTPUT("Failed to update inventory ." + str(e))
```

```
END Function
```

6.4 Display.py

```
Function display_inventory(inventory):
```

```
    Try:
```

```
        OUTPUT("{:^85}".format("Inventory"))
```

```
        OUTPUT("-"*85)
```

```
        OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format(
            "ID","Shoe", "Brand", "Available", "Price", "Source"))
```

```
        OUTPUT("-"*85)
```

```
        For row in inventory:
```

```
            OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(
                row[0], row[1], row[2], row[3], row[4], row[5]))
```

```
        END For
```

```
        OUTPUT("-"*85)
```

```
    Except Exception as e:
```

```
        OUTPUT("Failed to display Data: ", e)
```

```
    END Try
```

```
END Function
```

```
Function display_available(inventory):
```

```
    Try:
```

```
        OUTPUT("{:^85}".format("Inventory"))
```

```
        OUTPUT("-"*85)
```

```
        OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format(
            "ID","Shoe", "Brand", "Available", "Price", "Source"))
```

```
        OUTPUT("-"*85)
```

```

For row in inventory:
    If (row[3]>0):
        OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} |
        {:<14}".format(row[0], row[1], row[2], row[3], row[4], row[5]))
    END if
END For
OUTPUT("-"*85)
Except Exception as e:
    OUTPUT("Failed to display Data: ", e)
END Try
END Function

Function display_buylist (boughtlist,customername):
    Try:
        OUTPUT("{:^85}".format("CART FOR "+customername))
        OUTPUT("-"*85)
        OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format(
            "ID","Shoe", "Brand", "Available", "Price", "Source"))
        OUTPUT("-"*85)
        For row in boughtlist:
            OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(
                row[0], row[1], row[2], row[3], row[4], row[5]))
        END For
        OUTPUT("-"*85)
    Except Exception as e:
        OUTPUT("Failed to display Data: ", e)
    END Try
END Function

```

6.5 Generate_bill.py

```

Import datetime

```

Function generate_bill(customer_name,boughtlist,customer=True):

Success = False

Total,bulkdiscount,domesticdiscount,discountAmount,finalPrice =

bulkdiscountcheck(boughtlist)

randomid = random_id()

file_name = customer_name + "_" + randomid + ".txt"

Try:

With open("Bills/" + file_name, "w") as file:

file.write("=" * 85 + "\n")

file.write("{:^85}\n".format("SPEEDZWEAR WHOLESALE"))

file.write("{:^85}\n".format("KAPAN, KTM, NEPAL"))

IF customer:

file.write("\n{:^85}\n".format("CUSTOMER INVOICE"))

ELSE:

file.write("\n{:^85}\n".format("RESTOCK INVOICE"))

END IF

file.write("-" * 85 + "\n")

file.write("Bill ID : " + randomid + "\n")

file.write("Transaction Date: " + str(datetime.date.today()) + "\n")

IF customer:

file.write("Billed to : " + customer_name + "\n")

ELSE:

file.write("Supplier : " + customer_name + "\n")

END IF

file.write("-" * 85 + "\n")

file.write("{:<5} {:<15} {:<15} {:<10} {:<15} {:<15}\n".format(

"Sn", "Item", "Brand", "Qty", "Rate", "Amount"))

file.write("-" * 85 + "\n")

n=1

For row in boughtlist:

```
file.write("{:<5} {:<15} {:<15} {:<10} {:<15.2f} {:<15.2f}\n".format(
n, row[1], row[2],row[3],row[4],(row[3]*row[4])))
```

END For

n+=1

IF customer :

```
file.write("{:50}{:35}\n".format(" ","-"*35))
file.write("{:50}{:35}\n".format(" ","Discount Applied"))
file.write("{:50}{:20}{:<15.2f}\n".format(" ",
"Bulk Discount    :",bulkdiscount))
file.write("{:50}{:20}{:<15.2f}\n".format(" ",
"Domestic Discount :",domesticdiscount))
file.write("{:50}{:35}\n".format(" ","-"*35))
file.write("{:50}{:20}{:<15.2f}\n".format(" ",
"Gross amount    :",total))
file.write("{:50}{:20}{:<15.2f}\n".format(" ",
"Discount amount :",discountAmount))
file.write("{:50}{:20}{:<15.2f}\n".format(" ",
"Net amount      :",finalPrice))
file.write("-"*85+"\n")
file.write("{:^85}\n".format(
"EXCHANGE IN 7 DAYS WITH INVOICE BETWEEN 10-
8PM"))
```

ELSE:

```
file.write("{:50}{:35}\n".format(" ","-"*35))
file.write("{:50}{:20}{:<15.2f}\n".format(" ",
"Net amount      :",total))
file.write("-"*85+"\n")
```

```

        file.write("{:^85}\n".format(
            "Restocks are to be made by staff only"))

    END IF

    file.write("{:^85}\n".format("***CONDITIONS APPLY**"))
    file.write("=*85+\n")

    OUTPUT("\\nInvoice saved in Bills as {}".format(file_name))
    success = True

Except FileNotFoundError:
    OUTPUT("Could not access Bills folder.")

Except Exception as e:
    OUTPUT("Failed to generate Invoice {}".format(e))

END Try
    return success

END Function

Function randrom_id():
    now = datetime.datetime.now()
    month = str(now.month)
    minute = str(now.minute)
    second = str(now.second)
    microsecond = str(now.microsecond)
    random = month + minute + second + microsecond
    return random

END Function

Function bulkdiscountcheck(list1):
    product_total = 0
    bulk_total = 0
    domestic_product_total = 0

    For row in list1:

```

```

    amonunt = row[4]*row[3]
    product_total += amount
    IF row[3] > 10:
        bulk_total += amount
        IF row[5]=="Domestic":
            domestic_product_total += amount
        END IF
    END IF
END For

discountbulk = bulk_total*0.05
discountbulkdomeestic = domestic_product_total*0.07
discountAmount = discountbulk+discountbulkdomeestic
finalPrice = product_total - discountAmount

return product_total,discountbulk,discountbulkdomeestic,
discountAmount,finalPrice

```

END Function

6.6 Operations.py

FUNCTION customer_sell(inventory):

Try:

```

OUTPUT("=="*24)

```

While True:

```

    nameid = input("Enter Product name or id (or type EXIT to cancel): ")

```

```

    IF nameid.upper() == "EXIT":

```

```

        return None

```

```

    END IF

```

```

    row = find_product(inventory,nameid)

```

```

    IF not row:

```

```

        OUTPUT("Product not found")

```

```

        continue

```



```

END IF
break
OUTPUT("Product found")
print_row(row)
While True:
    units_purchased = int_input("\nUnits purchased
(or type EXIT to cancel): ")
    IF units_purchased == "EXIT":
        return None
    END IF
    IF (units_purchased > row[3]):
        OUTPUT("Not enough stock")
    ELSE:
        Break
    END IF
    row[3] -= units_purchased
    purchased_item = row.copy()
    purchased_item[3] = units_purchased
    OUTPUT("**Added product to cart**\n")
    return purchased_item
Except Exception as e:
    OUTPUT("Failed Operation {}".format(e))
END Try
END Function
Function restock_product(inventory):
    Try:
        boughtlist = []
        OUTPUT("=="*24)

```

```
supplier_name = input("Enter Supplier name (or type EXIT to cancel):")
```

```
IF supplier_name.upper() == "EXIT":
```

```
    return None, None
```

```
END IF
```

```
While True:
```

```
    nameid = input("Enter Product name or id
```

```
    (or type EXIT to cancel): ")
```

```
    IF nameid.upper() == "EXIT":
```

```
        return None, None
```

```
    END IF
```

```
    row = find_product(inventory, nameid)
```

```
    IF not row:
```

```
        OUTPUT("Product not found")
```

```
        continue
```

```
    OUTPUT("Product found")
```

```
    print_row(row)
```

```
    units_purchased = int_input("\nUnits purchased
```

```
    (or type EXIT to cancel): ")
```

```
    IF units_purchased == "EXIT":
```

```
        return None, None
```

```
    END IF
```

```
    purchased_price = float_input("\nPurchase Price :")
```

```
    While True:
```

```
        x = input("Update Selling price? (y / n)")
```

```
        x = x.lower()
```

```
        IF (x == "n"):
```

```
            break
```

```
        ELIF (x == "y"):
```

```

        OUTPUT("Old Price per-Unit :",row[4])
        new_price = int_input("ENter New selling price :")
        row[4] = new_price
        break
    ELSE:
        OUTPUT("\nInvalid Entry\n")
    END IF
    row[3] += units_purchased
    purchased_item = row.copy()
    purchased_item[3] = units_purchased
    purchased_item[4] = purchased_price
    boughtlist.append(purchased_item)
    x = input("Add another product? (y / n / or type EXIT to cancel )")
    x = x.lower()
    IF(x == "n"):
        break
    ELIF(x == "y"):
        continue
    ELIF(x=="exit"):
        return None,None
    ELSE:
        OUTPUT("\nInvalid Entry\n")
    END IF
    return supplier_name,boughtlist
Except Exception as e:
    OUTPUT("Failed Operation {}".format(e))
END Try
END Function

```

Function add_product(inventory):

Try:

boughtlist = []

OUTPUT("=="*24)

supplier_name = input("Enter Supplier name (or type EXIT to cancel):")

IF supplier_name.upper() == "EXIT":

return None, None

END IF

While True:

id = input("Enter product ID (or type EXIT to cancel):")

IF id.upper() == "EXIT":

return None, None

END IF

IF find_product(inventory, id):

OUTPUT("Product ID already exists")

Continue

END IF

name = input("Enter product name (or type EXIT to cancel):")

brand = input("Enter brand name (or type EXIT to cancel):")

stock = int_input("Enter Bought stock (or type EXIT to cancel):")

IF stock == "EXIT":

return None, None

END IF

boughtprice = float_input("Enter unit buying price :")

sellprice = float_input("Enter unit Selling price :")

While True:

source = input("Enter Source of manufacture
(Domestic/International) :")

source = source.title()

```

        IF(source != "Domestic" and source != "International"):
            OUTPUT("Wrong source")
            Continue
        END IF
        break
    boughtlist.append([id,name.title(),brand,stock,boughtprice,source])
    inventory.append([id,name.title(),brand,stock,sellprice,source])
    x = input("Add another product? (y / n / or type EXIT to cancel)")
    x = x.lower()
    IF(x == "n"):
        break
    EIF(x == "y"):
        continue
    ELIF(x=="exit"):
        return None,None
    ELSE:
        OUTPUT("\nInvalid Entry\n")
    END IF
    return supplier_name,boughtlist
Except Exception as e:
    OUTPUT("Failed Operation {}".format(e))
END Try
END Function
Function calceled_restock(inventory,boughtlist):
    Try:
        For item in boughtlist:
            product = find_product(inventory, item[0])
            IF product:

```

```

        product[3] += item[3]
    END IF
END For
Except Exception as e:
    OUTPUT("Failed to restock cancelled product {}".format(e))
END Try
END Function
Function find_product(inventory,nameid):
    Try:
        For row in inventory:
            IF row[0].lower() == nameid.lower() or
               row[1].lower() == nameid.lower():
                return row
            END IF
        END For
        return None
    Except Exception as e:
        OUTPUT("Failed Operation {}".format(e))
    END Try
END Function
Function int_input(string):
    While True:
        value = input(string)
        IF(value.upper()=="EXIT"):
            return "EXIT"
        END IF
    Try:
        value = int(value)

```

```

        IF (value<=0):
            print("Enter valid value")
        ELSE:
            return value
        END IF
    Except ValueError:
        OUTPUT("Please enter an valid value.")
    Except Exception as e:
        OUTPUT("Failed Operation {}".format(e))
    END Try
END Function

Function float_input(string):
    While True:
        value = input(string)
        Try:
            value = float(value)
            IF (value<=0):
                print("Enter valid value")
            ELSE:
                return value
            END IF
        Except ValueError:
            OUTPUT("Please enter an valid value.")
        Except Exception as e:
            OUTPUT("Failed Operation {}".format(e))
        END Try
    END Function

```

Function int_input(string):

```
    OUTPUT("="*85)
```

```
    OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format(
        "ID", "Shoe", "Brand", "Available", "Price", "Source"))
```

```
    OUTPUT("-"*85)
```

```
    OUTPUT("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format(
        row[0],row[1],row[2],row[3],row[4],row[5]))
```

END Function

7 Data Structures

This system uses both primitive and complex data structures to store and manipulate data as needed to perform operations for the program but also for manipulating txt files to read, write and generate new files. The data structure is divided into two categories:

7.1 Primitive data

The fundamental built-in data type this directly supported by the programming language is called primitive data type. These data types are not composed of other data types

7.1.1 String

String is the sequence of characters used to represent a text. Since python doesn't have a chr data type string can be considered a primitive data type. In this project String is used to store text data about product name, brand, id and other textual information. String data are identified enclosing text in quotations marks. It supports operations like concatenations (+), split and more. Only string can be concatenated to string. Functions like str() help convert data from float or integer into string making it able to use string operations on them. Some of the other Built-in functions commonly used are lower(), upper(), replace(), split(), format(), etc. The operation help manipulate string as per the need of user

Example:

```
"Hello world"
```

```
a = "Hello world"
```


Usage in project:

```
print("{:^85}".format("Inventory"))
print("="*85)
print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format("ID", "Shoe", "Brand", "Available", "Price", "Source"))
print("-"*85)
for row in inventory:
    print("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(row[0], row[1], row[2], row[3], row[4], row[5]))
print("="*85)
```

Figure 6-1 String Usage

Where all the highlighted text are Strings

7.1.2 Integer

Integer is a whole number without any decimal points. Integer data type stores integer values. In this project Integer is used to store numerical value such as available quantity, order amount and more. Integers are used for whole countable data. Basic arithmetic operations can be performed using this data type to produce new data of integer. It is also used as index values for String, List and Tuple. When arithmetic operation is done between an integer and float the resultant data type is float.

Example:

```
x = 12
```

```
print(10/5)
```

Usage in project:

```
print("{:^85}".format("Inventory"))
print("="*85)
print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format("ID", "Shoe", "Brand", "Available", "Price", "Source"))
print("-"*85)
for row in inventory:
    print("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(row[0], row[1], row[2], row[3], row[4], row[5]))
print("="*85)
```

Figure 6-2 Integer usage

Where the highlighted parts are integers

7.1.3 Float

Float is a number/integer that has decimal points. It is used to store precise numerical value when needed. When arithmetic operation are performed with this data type the output is always float i.e. when integer and float are used for arithmetic operation, the output is always float. In this project it is used to store product price and any other data that should have precise value to the decimal.

Example:

```
a = 2.5
```

```
x = 5.0/2
```

Usage in project:

```
with open("Catalogue.txt","r") as file:
    lines = file.readlines()
for line in lines:
    parts = line.replace("\n", "").split(",")
    parts[3]=int(parts[3])
    parts[4]=float(parts[4])
    result_list.append(parts)
```

Figure 6-3 Usage of float

Where the highlighted part is float (string from list is getting converted into float)

7.1.4 Boolean

Boolean is a data type that stores either True or False. This is very commonly used in conditioning logic. In this project it can be used for looping and function return value to check inventory availability. In Python 3 and above value of a Boolean variable can be changed anytime but the actual value cannot be changed because it is reserved i.e. True cannot be equal to False, whereas in Python 2. Booleans were not always reserved so it was possible.

Example:

```
isTrue = True
```

```
print(False)
```

Usage:

```

istrue = True
while istrue:
    ID = input("Enter id of student[Format:ID001] : ")
    for row in data:
        if ID == row[0]:
            print("ID already exists")
            break
        else:
            print("works")
            istrue = False
    return ID

```

Figure 6-4 Boolean usage

Where the highlighted text is Boolean

7.2 Complex Data Structures

Complex data type structure is those that are composed of multiple data types that work together to form a new type of data. Complex data structure in Python includes List, tuples, dictionary. These data types have their own methods to extract data from the structure, as a whole or singular. In this project complex data type is used to store values retrieved from TXT file and perform required operation with them

7.2.1 Tuples

Tuples are complex data types that are ordered, immutable i.e. they cannot be changed after creation. Tuples are used mainly when a function has to return multiple values. It can contain any type of data. The values stored in Tuples can be retrieved using index or direct assignment. Tuples have built-in methods such as `count()` and `index()`

Example:

```

x = (200,10)

print(x[0])

```

Usage:

```
fruits = ('apple', 'banana', 'orange')
for fruit in fruits:
    print(fruit)
```

Figure 6-5 Tuple usage

Where the highlighted part is Tuple

7.2.2 List

List is an ordered and mutable collection data type. It can contain any type of data. This include both primitive data and other collection type data like Tuples & Dictionary. The data contained can be accessed and modified using its index. Since this is an ordered collection data type the order remains as it is i.e. index of values do not change. List has several Built-in methods such as append(), Extend(), insert(), remove(), pop(), clear(), sort() and more. All these functions make List very useful as a collection data type to manipulate the data easily.

Example:

```
List1 = []
```

```
List2 = [2,3,4,7,8,4]
```

Usage in project:

```
def read_inventory():
    result_list=[]
    try:
        with open("Catalogue.txt","r") as file:
            lines = file.readlines()
            for line in lines:
                parts = line.replace("\n", "").split(",")
                parts[3]=int(parts[3])
                parts[4]=float(parts[4])
                result_list.append(parts)
            return result_list
    except FileNotFoundError:
        print("File not found")
```

Figure 6-6 Usage of list

Where the highlighted parts are list (Creating a 2d list)

7.2.3 Dictionary

Dictionary is an ordered and mutable collection data type. It is similar to list but instead of accessing data using index, this uses keys to access the values. Here keys are immutable but the values are mutable. Keys are made by the programmer and it can be created and deleted. Just like List, the order depends on the order in which the data was added. Dictionary also has a wide range of Built-in functions such as keys(), values(), items(), pop(), update(), and more. This makes accessing and manipulating keys and values easier.

Example:

```
Dic = {}
```

Usage:

```
newdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(newdict)|
```

Figure 6-7 Usage of dictionary

Where the highlighted parts are dictionary

7.2.4 Set

Set is an unordered and mutable collection data type. This doesn't allow duplicate data entry and the order of entry doesn't matter. This data type can perform set operations such as `intersection()`, `difference()`, `union()`. Here values aren't accessed by index, hence the whole set is to be access. This can be useful to remove redundant data hence it is useful for uniqueness. Other then the set operations there are also `add()`, `update()`, `remove()`, `clear()`, `pop()`, `copy()`, and more. As always these built-in functions make the data structure more functional able without having to make our own function to do these operations. Unlike other data types we cannot create empty set.

Example:

```
a = {1,2,3}
```

```
b = {2,3,5}
```

Usage:

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}
union_set = set_a | set_b # {1, 2, 3, 4, 5, 6}
intersection_set = set_a & set_b # {3, 4}
difference_set = set_a - set_b # {1, 2}
```

Figure 6-8 Usage of set

Where the highlighted parts are set and set operations

8 Execution of Porgram

Given below are Titles of operation and screenshot of program running the operation with valid user input and their respective outputs

8.1 Display Inventory

This operation is to display the current Inventory of the store. It displays all the items that are in stock even if there are no stock available.

```
*****
*                                *
*              OPERATIONS        *
*                                *
*  1. Display Inventory  2. Stock product  *
*                                *
*  3. Sell Product      4. EXIT          *
*                                *
*****

Select Operation : 1

                               Inventory
=====
ID   | Shoe           | Brand       | Available | Price      | Source
-----|-----|-----|-----|-----|-----
0X1  | Inigo 732      | Caliber     | 0         | Rs. 2700.0 | Domestic
0Y2  | Lite Racer     | Adidas      | 50        | Rs. 7000.0 | International
0Z3  | Loafer Light   | GoldStar    | 200       | Rs. 1000.0 | Domestic
0S4  | LENNOX 725     | Caliber     | 50        | Rs. 3650.89 | Domestic
0X2  | Ava Rover      | Nike        | 0         | Rs. 9680.99 | International
0x3  | Juta           | JUUtaaa     | 200       | Rs. 4000.0 | International
=====

*****
*                                *
*              OPERATIONS        *
*                                *
*  1. Display Inventory  2. Stock product  *
*                                *
*  3. Sell Product      4. EXIT          *
*                                *
*****

Select Operation : |
```

Figure 7-1 Display Inventory

8.2 Stock product

This Operations opens another menu / Option panel that shows operations related to product Stock. This is to be used by Staff to add or restock product

```

*****
*                                *
*              OPERATIONS        *
*                                *
*  1. Display Inventory  2. Stock product  *
*                                *
*  3. Sell Product      4. EXIT          *
*                                *
*****

Select Operation : 2

*****
*                                *
*          STOCK OPERATIONS      *
*                                *
*  1. ADD new Product    2. Restock product  *
*                                *
*  3. BACK                *
*                                *
*****

Select Operation : |

```

Figure 7-2 Stock Product

8.2.1 Add new Product

This operation is to add new product to the inventory. For this the user needs to input product id which doesn't already exists in inventory and enter valid details of product. User can add multiple product at once from a single supplier and create invoice.

```

*****
*                                *
*          STOCK OPERATIONS      *
*                                *
*  1. ADD new Product    2. Restock product  *
*                                *
*  3. BACK                *
*                                *
*****

Select Operation : 1
=====
Enter Supplier name (or type EXIT to cancel):Raunit
Enter product ID (or type EXIT to cancel):xxx
Enter product name (or type EXIT to cancel):Juta
Enter brand name (or type EXIT to cancel):Islington
Enter Bought stock (or type EXIT to cancel):200
Enter unit buying price :18000
Enter unit Selling price :20000
Enter Source of manufacture (Domestic/International) :Domestic
Add another product? (y / n / or type EXIT to cancel)n

Invoice saved in Bills as Raunit_85322952164.txt

*****
*                                *
*          STOCK OPERATIONS      *
*                                *
*  1. ADD new Product    2. Restock product  *
*                                *
*  3. BACK                *
*                                *
*****

Select Operation : |

```

Figure 7-3 Add new Product


```

=====
                        JUTA PASAL WHOLESALE
                        KAPAN, KTM, NEPAL

                        RESTOCK INVOICE
=====
Bill ID       : 84815155004
Transaction Date: 2025-08-22
Supplier      : Raunit
=====
Sn   Item      Brand      Qty      Rate      Amount
-----
1    Juta      Islington  200      18000.00  3600000.00
=====
Net amount      : 3600000.00
=====
Restocks are to be made by staff only
**CONDITIONS APPLY**
=====

```

Figure 7-4 Invoice for Add new Product

8.2.2 Restock product

This operation is to restock product to the inventory. For this the user needs to input product id which already exists in inventory and enter valid details of product. User can restock multiple product at once from a single supplier and create invoice.

```

*****
*          STOCK OPERATIONS          *
*          *                          *
* 1. ADD new Product  2. Restock product *
*          *                          *
* 3. BACK              *
*          *                          *
*****

Select Operation : 2

Inventory
=====
ID | Shoe      | Brand      | Available | Price      | Source
-----
0X | Inigo 732 | Caliber    | 200       | Rs. 2700.0 | Domestic
0Y | Lite Racer | Adidas     | 50        | Rs. 7000.0 | International
0Z | Loafer Light | GoldStar  | 150       | Rs. 1000.0 | Domestic
0S | LENNOX 725 | Caliber    | 50        | Rs. 3650.89 | Domestic
0B | Ava Rover | Nike       | 0         | Rs. 9680.99 | International
01 | Juta      | Islington  | 210       | Rs. 10000.0 | Domestic
xxx | Juta      | Islington  | 200       | Rs. 20000.0 | Domestic
=====

Enter Supplier name (or type EXIT to cancel):Islington
Enter Product name or id (or type EXIT to cancel): 0b
Product found
=====
ID | Shoe      | Brand      | Available | Price      | Source
-----
0B | Ava Rover | Nike       | 0         | 9680.99    | International

Units purchased (or type EXIT to cancel): 200

Purchase Price :2000
Update Selling price? (y / n)n
Add another product? (y / n / or type EXIT to cancel )n

Invoice saved in Bills as Islington_85455481198.txt

*****
*          STOCK OPERATIONS          *
*          *                          *
* 1. ADD new Product  2. Restock product *
*          *                          *
* 3. BACK              *
*          *                          *
*****

Select Operation : |

```

Figure 7-5 Restock Product

```

=====
JUTA PASAL WHOLESALE
KAPAN, KTM, NEPAL

RESTOCK INVOICE
-----
Bill ID      : 83829895380
Transaction Date: 2025-08-22
Supplier     : Islington
-----
Sn  Item      Brand      Qty      Rate      Amount
-----
1   Inigo 732   Caliber    200      2000.00    400000.00
-----
Net amount      : 400000.00
-----
Restocks are to be made by staff only
**CONDITIONS APPLY**
=====

```

Figure 7-6 Invoice for Restock Product

8.3 Sell product

This Operation opens another menu/option panel that shows operations related to selling products after user enter the name of the customer. This is to be used when a customer makes a purchase.

```

*****
*                                *
*                      OPERATIONS                      *
*                                *
*  1. Display Inventory  2. Stock product                *
*                                *
*  3. Sell Product       4. EXIT                        *
*                                *
*****

Select Operation : 3
Enter Customer Name : Raunit

*****
*                                *
*                      Customer OPERATIONS              *
*                                *
*  1.Show Cart           2.Add product                  *
*                                *
*  3.Buy and Generate Bill  4.Cancel purchase            *
*                                *
*****

Select Operation : |

```

Figure 7-7 Sell Product

8.3.1 Add product to cart

This operation requires valid product Id and units sold from the user after which the product is added to a cart (list) for that customer.

```
Enter Customer Name : Raunit
*****
*           Customer OPERATIONS           *
*                                         *
* Customer Name : Raunit                  *
*                                         *
* 1.Show Cart          2.Add product       *
*                                         *
* 3.Buy and Generate Bill  4.Cancel purchase *
*                                         *
*****

Select Operation : 2

Inventory
=====
ID | Shoe | Brand | Available | Price | Source
-----
OX | Inigo 732 | Caliber | 200 | Rs. 2700.0 | Domestic
OY | Lite Racer | Adidas | 50 | Rs. 7000.0 | International
OZ | Loafer Light | GoldStar | 150 | Rs. 1000.0 | Domestic
OS | LENNOX 725 | Caliber | 50 | Rs. 3650.99 | Domestic
OB | Ava Bover | Nike | 200 | Rs. 9680.99 | International
OI | Juta | Islington | 210 | Rs. 10000.0 | Domestic
xxx | Juta | Islington | 200 | Rs. 20000.0 | Domestic
=====

Enter Product name or id (or type EXIT to cancel): OZ
Product found
=====
ID | Shoe | Brand | Available | Price | Source
-----
OZ | Loafer Light | GoldStar | 150 | 1000.0 | Domestic

Units purchased (or type EXIT to cancel): 50

**Added product to cart**

*****
*           Customer OPERATIONS           *
*                                         *
* Customer Name : Raunit                  *
*                                         *
* 1.Show Cart          2.Add product       *
*                                         *
* 3.Buy and Generate Bill  4.Cancel purchase *
*                                         *
*****

Select Operation :
```

Figure 7-8 Add product to cart

8.3.2 Show Cart

Displays cart in a table which hold all the products added by the add product operation for that user.

```
*****
*           Customer OPERATIONS           *
*                                         *
* 1.Show Cart          2.Add product       *
*                                         *
* 3.Buy and Generate Bill  4.Cancel purchase *
*                                         *
*****

Select Operation : 1

Inventory
=====
ID | Shoe | Brand | QTY | Price | Source
-----
OZ3 | Loafer Light | GoldStar | 50 | Rs. 1000.0 | Domestic
=====

*****
*           Customer OPERATIONS           *
*                                         *
* 1.Show Cart          2.Add product       *
*                                         *
* 3.Buy and Generate Bill  4.Cancel purchase *
*                                         *
*****

Select Operation : |
```

Figure 7-9 Display Cart

8.3.3 Buy and Generate Bill

This operation finalizes the customer interaction and generated invoice in .txt format that is stored in Bills folder. The generate bill will account for all discounts that can be applied.

```
*****
*                               *
*           Customer OPERATIONS *
*                               *
* 1.Show Cart           2.Add product *
*                               *
* 3.Buy and Generate Bill 4.Cancel purchase *
*                               *
*****

Select Operation : 3

Invoice saved in Bills as Raunit_8448716536.txt

*****
*                               *
*           OPERATIONS          *
*                               *
* 1. Display Inventory 2. Stock product *
*                               *
* 3. Sell Product      4. EXIT      *
*                               *
*****

Select Operation : |
```

Figure 7-10 Buy and Generate Bill

=====

JUTA PASAL WHOLESALE

KAPAN, KTM, NEPAL

=====

CUSTOMER INVOICE

Bill ID : 8448716536

Transaction Date: 2025-08-22

Billed to : Raunit

Sn	Item	Brand	Qty	Rate	Amount
1	Loafer Light	GoldStar	50	1000.00	50000.00

Discount Applied	
Bulk Discount	: 2500.00
Domestic Discount	: 3500.00

Gross amount	: 50000.00
Discount amount	: 6000.00
Net amount	: 44000.00

EXCHANGE IN 7 DAYS WITH INVOICE BETWEEN 10-8PM

CONDITIONS APPLY

=====

Figure 7-11 Invoice for Customer

8.4 EXIT

Exits the program for that session.

```
*****
*                               *
*           OPERATIONS          *
*                               *
* 1. Display Inventory 2. Stock product *
*                               *
* 3. Sell Product      4. EXIT      *
*                               *
*****

Select Operation : 4
EXIT PROGRAM
```

Figure 7-12 Exit

9 Testing

9.1 Test 1

Objective	Show implementation of try, except
Action	Provide invalid input, the input is to be checked and converted by helper function which will create an exception
Expected Result	The program should catch the exception, show appropriate message and performs task
Actual Result	The program catches the exception, shows appropriate message and performs task
Conclusion	Test successful

Table 1 Test 1 Table

```
while True:
    units_purchased = int_input("\nUnits purchased (or type EXIT to cancel): ")
    if units_purchased == "EXIT":
        return None
    if(units_purchased > row[3]):
        print("Not enough stock")
    else:
        break
```

Figure 8-1 Test 1 try, catch input field

```
def int_input(string):
    """
    description: Helper function for invalid and exception check
    parameter : String prompt for input
    return type: int for valid input
    """
    while True:
        value = input(string)
        if(value.upper()=="EXIT"):
            return "EXIT"
        try:
            value = int(value)
            if(value<=0):
                print("Enter valid value")
            else:
                return value
        except ValueError:
            print("Please enter an valid value.")
        except Exception as e:
            print("Failed Operation {}".format(e))
```

Figure 8-2 Test 1 try, catch helper function

```
Units purchased (or type EXIT to cancel): xx
Please enter an valid value.
```

```
Units purchased (or type EXIT to cancel): 12
```

```
**Added prodcut to cart**
```

Figure 8-3 Test 1 try, catch Output

9.2 Test 2

Objective	Selection purchase and sale of products
Action	Provide negative value as input and also provide non existed value in product id search block
Expected Result	Negative value should not be accepted and the user should be asked to re-enter value, Non-existent product id / name should result in user being asked to re-enter value
Actual Result	Negative value was not accepted and the user re-enter value, Non-existent product id / name resulted in user being asked to re-enter value
Conclusion	Test successful

Table 2 Test 2 Table

```

while True:
    units_purchased = int_input("\nUnits purchased (or type EXIT to cancel): ")
    if units_purchased == "EXIT":
        return None
    if(units_purchased > row[3]):
        print("Not enough stock")
    else:
        break

```

Figure 8-4 Test 2 Negative input : input field code

```

def int_input(string):
    """
    description: Helper function for invalid and exception check
    parameter : String prompt for input
    return type: int for valid input
    """
    while True:
        value = input(string)
        if(value.upper()=="EXIT"):
            return "EXIT"
        try:
            value = int(value)
            if(value<=0):
                print("Enter valid value")
            else:
                return value
        except ValueError:
            print("Please enter an valid value.")
        except Exception as e:
            print("Failed Operation {}".format(e))

```

Figure 8-5 Test 2 Negative input : value check

```

Units purchased (or type EXIT to cancel): -12
Enter valid value

Units purchased (or type EXIT to cancel): 12

**Added prodcut to cart**

```

Figure 8-6 Test 2 Negative input: Output

```

while True:
    nameid = input("Enter Product name or id (or type EXIT to cancel): ")
    if nameid.upper() == "EXIT":
        return None
    row = find_product(inventory,nameid)
    if not row:
        print("Product not found")
        continue
    break
print("Product found")

```

Figure 8-7 Test 2 Non- exist value : input field code

```

def find_product(inventory, nameid):
    """
    description: Helper function for product check
    parameter : list that has inventory data, product id
    return type: list of required product
    """
    try:
        for row in inventory:
            if row[0].lower() == nameid.lower() or row[1].lower() == nameid.lower():
                return row
        return None
    except Exception as e:
        print("Failed Operation {}".format(e))

```

Figure 9-8 Test 2 Non-exist value : value check

```

Enter Product name or id (or type EXIT to cancel): xx
Product not found
Enter Product name or id (or type EXIT to cancel): 0y
Product found

```

Figure 8-9 Non-exist: Output

9.3 Test 3

Objective	File generation of purchase of product(s) (Purchasing multiple product(s))
Action	From main menu direct to stock Operation menu, add/ restock 2-3 products from add/restock product.
Expected Result	Invoice should be generated in Bills folder with a success message
Actual Result	Invoice was generated in Bills folder with a success message
Conclusion	Test successful

Table 3 Test 3 Table


```
=====
JUTA PASAL WHOLESALE
KAPAN, KTM, NEPAL

RESTOCK INVOICE
-----
Bill ID      : 85248705189
Transaction Date: 2025-08-27
Supplier     : Raunit
-----
Sn  Item          Brand      Qty    Rate      Amount
-----
1  LENNOX 725    Caliber    20    3000.00   60000.00
2  Loafer Light  GoldStar  50    500.00   25000.00
-----
Net amount      : 85000.00
-----

Restocks are to be made by staff only
**CONDITIONS APPLY**
=====
```

Figure 8-12 Test 3 Restock product Invoice

9.4 Test 4

Objective	File generation of sales process of product(s) (Selling multiple product(s))
Action	From main menu direct to customer Operation menu, add 2-3 products to cart from add product then select option to buy product and generate bill to finalize purchase
Expected Result	Invoice should be generated in Bills folder with a success message
Actual Result	Invoice was generated in Bills folder with a success message
Conclusion	Test successful

Table 4 Test 4 Table

```

*****
*                               *
*               OPERATIONS      *
*                               *
*  1. Display Inventory  2. Stock product  *
*                               *
*  3. Sell Product      4. EXIT      *
*                               *
*****

Select Operation : 3
Enter Customer Name : Raunit

*****
*                               *
*       Customer OPERATIONS    *
*                               *
*  1.Show Cart              2.Add product  *
*                               *
*  3.Buy and Generate Bill  4.Cancel purchase *
*                               *
*****

Select Operation : |

```

Figure 8-13 Test 4 Customer menu

```

*****
*                               *
*       Customer OPERATIONS    *
*                               *
* Customer Name : Raunit      *
*                               *
*  1.Show Cart              2.Add product  *
*                               *
*  3.Buy and Generate Bill  4.Cancel purchase *
*                               *
*****

Select Operation : 2

                               Inventory
=====
ID  | Shoe          | Brand    | Available | Price    | Source
-----
0X  | Inigo 732     | Caliber  | 180       | Rs. 2700.0 | Domestic
0Y  | Lite Racer   | Adidas   | 50        | Rs. 7000.0 | International
0Z  | Loafer Light  | GoldStar | 200       | Rs. 1000.0 | Domestic
0S  | LENNOX 725   | Caliber  | 20        | Rs. 3650.89 | Domestic
0B  | Ava Rover    | Nike     | 200       | Rs. 9680.99 | International
=====

Enter Product name or id (or type EXIT to cancel): 0b
Product found
=====
ID  | Shoe          | Brand    | Available | Price    | Source
-----
0B  | Ava Rover    | Nike     | 200       | 9680.99  | International

Units purchased (or type EXIT to cancel): 10
**Added product to cart**

```

Figure 8-14 Test 4 Adding product to cart (1)

```

*****
*                               *
*           Customer OPERATIONS *
*                               *
* Customer Name : Raunit        *
*                               *
* 1.Show Cart           2.Add product *
*                               *
* 3.Buy and Generate Bill  4.Cancel purchase *
*                               *
*****

Select Operation : 2

                               Inventory
=====
ID   | Shoe           | Brand      | Available | Price   | Source
-----
0X   | Inigo 732      | Caliber    | 200       | Rs. 2700.0 | Domestic
0Y   | Lite Racer     | Adidas     | 50        | Rs. 7000.0 | International
0Z   | Loafer Light   | GoldStar   | 200       | Rs. 1000.0 | Domestic
0S   | LENNOX 725     | Caliber    | 20        | Rs. 3650.89 | Domestic
0B   | Ava Rover      | Nike       | 200       | Rs. 9680.99 | International
=====

Enter Product name or id (or type EXIT to cancel): 0x
Product found
=====
ID   | Shoe           | Brand      | Available | Price   | Source
-----
0X   | Inigo 732      | Caliber    | 200       | 2700.0   | Domestic

Units purchased (or type EXIT to cancel): 20

**Added prodcut to cart**

```

Figure 8-15 Test 4 Adding product to cart (2)

```

*****
*                               *
*           Customer OPERATIONS *
*                               *
* Customer Name : Raunit        *
*                               *
* 1.Show Cart           2.Add product *
*                               *
* 3.Buy and Generate Bill  4.Cancel purchase *
*                               *
*****

```

Select Operation : 3

Invoice saved in Bills as Raunit_85838369986.txt

Figure 8-16 Test 4 Buy and Generate bill

```

=====
JUTA PASAL WHOLESALE
KAPAN, KTM, NEPAL

CUSTOMER INVOICE
-----
Bill ID       : 85838369986
Transaction Date: 2025-08-27
Billed to    : Raunit
-----
Sn  Item      Brand    Qty    Rate    Amount
-----
1   Inigo 732  Caliber  20     2700.00  54000.00
2   Ava Rover  Nike     10     9680.99  96809.90
-----
Discount Applied
Bulk Discount   : 2700.00
Domestic Discount : 3780.00
-----
Gross amount    : 150809.90
Discount amount : 6480.00
Net amount      : 144329.90
-----
EXCHANGE IN 7 DAYS WITH INVOICE BETWEEN 10-8PM
**CONDITIONS APPLY**
=====

```

Figure 8-17 Test 4 Customer invoice

9.5 Test 5

Objective	Show the update in stock of product(s)
Action	From main menu direct to customer Operation menu, add 2-3 products to cart from add product then select option to buy product and generate bill to finalize purchase (TEST 4) From main menu direct to stock Operation menu, add/ restock 2-3 products from add/restock product (TEST 3)
Expected Result	There should be changes in stock of product
Actual Result	There was changes in stock of product
Conclusion	Test successful

Table 5 Test 5 Table

```

0X,Inigo 732,Caliber,200,2700.0,Domestic
0Y,Lite Racer,Adidas,50,7000.0,International
0Z,Loafer Light,GoldStar,150,1000.0,Domestic
0S,LENNOX 725,Caliber,0,3650.89,Domestic
0B,Ava Rover,Nike,200,9680.99,International

```

Figure 8-18 Test 5: Inventory before Test 3

```

0X,Inigo 732,Caliber,200,2700.0,Domestic
0Y,Lite Racer,Adidas,50,7000.0,International
0Z,Loafer Light,GoldStar,200,1000.0,Domestic
0S,LENNOX 725,Caliber,20,3650.89,Domestic
0B,Ava Rover,Nike,200,9680.99,International

```

Figure 8-19 Test 5: Inventory after Test 3

```

0X,Inigo 732,Caliber,200,2700.0,Domestic
0Y,Lite Racer,Adidas,50,7000.0,International
0Z,Loafer Light,GoldStar,200,1000.0,Domestic
0S,LENNOX 725,Caliber,20,3650.89,Domestic
0B,Ava Rover,Nike,200,9680.99,International

```

Figure 8-20 Test 5: Inventory before Test 4

```

0X,Inigo 732,Caliber,180,2700.0,Domestic
0Y,Lite Racer,Adidas,50,7000.0,International
0Z,Loafer Light,GoldStar,200,1000.0,Domestic
0S,LENNOX 725,Caliber,20,3650.89,Domestic
0B,Ava Rover,Nike,190,9680.99,International

```

Figure 8-21 Test 5: Inventory after Test 4

10 Conclusion

With this project I have successfully developed a management software for a shoe wholesaler. This system can track inventory; update it as required and add new items when needed. It also has operations to show all inventory and to show only available items in inventory. It can also manage transactions for both customer and seller i.e. restock transaction and selling transaction. These transactions also produce invoice that are saved in Bills folder which is alongside the program script. The invoice for customer also has discount feature that auto checks the condition for discount and adds discounts when needed. Overall, this project helped demonstrate python's file handling and data structures and how it can be applied to real-world problems.

11 Appendix

11.1 Main.py

```
import Read
```

```
import Display
```

```
import Operations
```

```
import Write
```

```
from Generate_bill import generate_bill
```

```
def customer_operation(inventory):
```

```
    """
```

```
        description: Function that displays options for customer operations to add product to  
        cart, show cart and generate bill
```

```
            each option calls for specific function related to its operation
```

```
        parameter : list that holds the inventory data
```

```
        return type: none, return without value to the main menu
```

```
    """
```

```
    try:
```

```
        boughtlist=[]
```

```
        customername = input("Enter Customer Name : ")
```

```
        while True:
```

```

print("\n"+"***25)
print("{:^48}".format("Customer OPERATIONS"))
print("***+"+"*48+"")
print("** {:<44} {}".format("Customer Name : "+customername))
print("***+"+"*48+"")
print("** {:<25} {:<20}{}".format("1.Show Cart ","2.Add product"))
print("***+"+"*48+"")
print("** {:<25} {:<20}{}".format("3.Buy and Generate Bill","4.Cancel purchase"))
print("***+"+"*48+"")
print("***25+"\n")
selected = input("\nSelect Operation : ")
if(selected == "1" ):
    if boughtlist:
        Display.display_buylist(boughtlist)
    else:
        print("\n**Empty Cart**\n")
elif (selected == "2"):
    Display.display_available(inventory)
    purchased_item = Operations.customer_sell(inventory)
    if purchased_item:
        boughtlist.append(purchased_item)
elif (selected == "3"):
    if boughtlist:
        success = generate_bill(customername,boughtlist)
        if success:
            Write.write_inventory(inventory)
        else:
            print("Purchase Cancelled")

```

```

        break
    else:
        print("\nEmpty Cart\n")
    elif(selected == "4"):
        if boughtlist:
            Operations.canceled_restock(inventory,boughtlist)
            print("\n**Purchase Cancelled**\n")
            break
        else:
            print("\nInvalid Entry\n")
    except Exception as e:
        print("Failed Operation {}".format(e))

```

```
def stock_operation(inventory):
```

```
    """
```

description: Function that displays options for inventory stock operations to add product to inventory, restock product

each options calls for specific function related to its operation

parameter : list that holds the inventory data

return type: none, return without value to the main menu

```
    """
```

```
    try:
```

```
        while True:
```

```
            print("\n"+"*****23)
```

```
            print("{}^{44}{}".format("STOCK OPERATIONS"))
```

```
            print("{}+ "44+"")
```

```
            print("{} {:<21} {:<20}{}".format("1. ADD new Product","2. Restock product"))
```

```
            print("{}+ "44+"")
```

```
            print("{} {:<21} {:<20}{}".format("3. BACK",""))
```

```

print("**"+" "*44+"**")
print("*****23+\n")
selected = input("\nSelect Operation : ")
if(selected == "1" ):
    suppliername,boughtlist = Operations.add_product(inventory)
    if boughtlist:
        success = generate_bill(suppliername,boughtlist,False)
        if success:
            Write.write_inventory(inventory)
elif (selected == "2"):
    Display.display_inventory(inventory)
    suppliername,boughtlist = Operations.restock_product(inventory)
    if boughtlist:
        success = generate_bill(suppliername,boughtlist,False)
        if success:
            Write.write_inventory(inventory)
elif (selected == "3"):
    print("\n**Back to main menu**\n")
    break
else:
    print("\nInvalid Entry\n")
except Exception as e:
    print("Failed Operation {}".format(e))

```

description: main code that runs right after program execution and keeps looping until the program ends

is the main menu of the program

"""

```
inventory_data = Read.read_inventory()
```

```
while True:
```

```
    try:
```

```
        print("\n"+"*****23)
```

```
        print("**{:^44}*".format("OPERATIONS"))
```

```
        print("**"+" "*44+"**")
```

```
        print("** {:<21} {:<20}*".format("1. Display Inventory","2. Stock product"))
```

```
        print("**"+" "*44+"**")
```

```
        print("** {:<21} {:<20}*".format("3. Sell Product","4. EXIT"))
```

```
        print("**"+" "*44+"**")
```

```
        print("*****23+"\n")
```

```
        selected = input("\nSelect Operation : ")
```

```
        if(selected == "1"):
```

```
            Display.display_inventory(inventory_data)
```

```
        elif (selected == "3"):
```

```
            customer_operation(inventory_data)
```

```
        elif (selected == "2"):
```

```
            stock_operation(inventory_data)
```

```
        elif (selected == "4"):
```

```
            print("\n**EXIT PROGRAM**\n")
```

```
            break
```

```
        else:
```

```
            print("\nInvalid Entry\n")
```

```
    except Exception as e:
```

```
        print("Failed Operation {}".format(e))
```

11.2 GenerateBill.py

```
import datetime

def generate_bill(customer_name,boughtlist,customer=True):

    success = False

    total, bulkdiscount, domesticdiscount, discountAmount,finalPrice =
bulkdiscountcheck(boughtlist)

    randomid = random_id()

    file_name = customer_name+"_"+randomid+".txt"

    try:

        with open("Bills/"+file_name,"w") as file:

            file.write("="*85+"\n")

            file.write("{:^85}\n".format("SPEEDZWEAR WHOLESALE"))

            file.write("{:^85}\n".format("KAPAN, KTM, NEPAL"))

            if customer:

                file.write("\n{:^85}\n".format("CUSTOMER INVOICE"))

            else:

                file.write("\n{:^85}\n".format("RESTOCK INVOICE"))

            file.write("-"*85+"\n")

            file.write("Bill ID      : "+randomid+"\n")

            file.write("Transaction Date: "+ str(datetime.date.today())+"\n")

            if customer:

                file.write("Billed to      : "+customer_name+"\n")

            else:

                file.write("Supplier       : "+customer_name+"\n")

            file.write("-"*85+"\n")

            file.write("{:<5} {:<15} {:<15} {:<10} {:<15} {:<15}\n".format("Sn", "Item",
"Brand", "Qty", "Rate", "Amount"))

            file.write("-"*85+"\n")

            n=1
```

```

    for row in boughtlist:

        file.write("{:<5} {:<15} {:<15} {:<10} {:<15.2f} {:<15.2f}\n".format(n, row[1],
row[2],row[3],row[4],(row[3]*row[4])))

        n+=1

    if customer :

        file.write("{:50}{:35}\n".format(" ","-"*35))

        file.write("{:50}{:35}\n".format(" ","Discount Applied"))

        file.write("{:50}{:20}{:<15.2f}\n".format(" ","Bulk Discount   :",bulkddiscount))

        file.write("{:50}{:20}{:<15.2f}\n".format(" ","Domestic Discount :
",domesticdiscount))

        file.write("{:50}{:35}\n".format(" ","-"*35))

        file.write("{:50}{:20}{:<15.2f}\n".format(" ","Gross amount   :",total))

        file.write("{:50}{:20}{:<15.2f}\n".format(" ","Discount amount :
",discountAmount))

        file.write("{:50}{:20}{:<15.2f}\n".format(" ","Net amount     :",finalPrice))

        file.write("-"*85+"\n")

        file.write("{:^85}\n".format("EXCHANGE IN 7 DAYS WITH INVOICE
BETWEEN 10-8PM"))

    else:

        file.write("{:50}{:35}\n".format(" ","-"*35))

        file.write("{:50}{:20}{:<15.2f}\n".format(" ","Net amount     :",total))

        file.write("-"*85+"\n")

        file.write("{:^85}\n".format("Restocks are to be made by staff only"))

        file.write("{:^85}\n".format("***CONDITIONS APPLY***"))

        file.write("="*85+"\n")

    print("\nInvoice saved in Bills as {}\n".format(file_name))

    success = True

except FileNotFoundError:

    print("Could not access Bills folder.")

except Exception as e:

```

```
    print("Failed to generate Invoice {}".format(e))
return success
```

```
def random_id():
    now = datetime.datetime.now()
    month = str(now.month)
    minute = str(now.minute)
    second = str(now.second)
    microsecond = str(now.microsecond)
    random = month + minute + second + microsecond
    return random
```

```
def bulkdiscountcheck(list1):
    product_total = 0
    bulk_total = 0
    domestic_product_total = 0
    for row in list1:
        amount = row[4]*row[3]
        product_total += amount
        if row[3] > 10:
            bulk_total += amount
            if row[5]=="Domestic":
                domestic_product_total += amount
    discountbulk = bulk_total*0.05
    discountbulkdomeestic = domestic_product_total*0.07
    discountAmount = discountbulk+discountbulkdomeestic
    finalPrice = product_total - discountAmount
```

```
return product_total, discountbulk, discountbulkdomestic, discountAmount, finalPrice
```

11.3 Display.py

```
def display_inventory(inventory):
```

```
    try:
```

```
        print("{:^85}".format("Inventory"))
```

```
        print("="*85)
```

```
        print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format("ID", "Shoe", "Brand",  
"Available", "Price", "Source"))#string format forrow of column titles
```

```
        print("-"*85)
```

```
        for row in inventory:
```

```
            print("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(row[0], row[1],  
row[2], row[3], row[4], row[5]))#string format to add row, taking data from single list  
            using index
```

```
        print("="*85)
```

```
    except Exception as e:
```

```
        print("Failed to display Data:", e)
```

```
def display_available(inventory):
```

```
    try:
```

```
        print("{:^85}".format("Inventory"))
```

```
        print("="*85)
```

```
        print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format("ID", "Shoe", "Brand",  
"Available", "Price", "Source"))#string format forrow of column titles
```

```
        print("-"*85)
```

```
        for row in inventory:
```

```
            if(row[3]>0):
```

```
                print("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(row[0], row[1],  
row[2], row[3], row[4], row[5]))#string format to add row, taking data from single list  
                using index
```

```
            print("="*85)
```

```
    except Exception as e:
```

```

print("Failed to display Data:", e)

def display_buylist(boughtlist, customername):
    try:
        print("{:^85}".format("CART FOR "+customername))
        print("="*85)
        print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format("ID", "Shoe", "Brand", "
QTY", "Price", "Source"))#string format forrow of column titles
        print("-"*85)
        for row in boughtlist:
            print("{:<5} | {:<15} | {:<12} | {:<10} | Rs. {:<10} | {:<14}".format(row[0], row[1],
row[2], row[3], row[4], row[5]))#string format to add row, taking data from single list
            using index
        print("="*85)
    except Exception as e:
        print("Failed to display Data:", e)

```

11.4 Read.py

```

def read_inventory():
    """
    description: Function that reads the data stored in file and modifies them into useable
    format into a list

    parameter : none as file name and path is fixed
    return type: list that contains the file data
    """
    result_list=[]
    try:
        with open("Catalogue.txt","r") as file:
            lines = file.readlines()
            for line in lines:
                parts = line.replace("\n","").split(",")

```

```

        parts[3]=int(parts[3])
        parts[4]=float(parts[4])
        result_list.append(parts)
    return result_list
except FileNotFoundError:
    print("File not found")
except Exception as e:
    print("Failed Operation : " + str(e))

```

11.5 Operation.py

```
def customer_sell(inventory):
```

```
    """
```

description: Function that searches for product that the customer wants and updates inventory table stock if that item is added to the cart

also check for invalid input and stock available

parameter : list that has inventory data

return type: list that contains product the the customer added to cart,

```
    """
```

```
    try:
```

```
        print("=="*24)
```

```
        while True:
```

```
            nameid = input("Enter Product name or id (or type EXIT to cancel): ")
```

```
            if nameid.upper() == "EXIT":
```

```
                return None
```

```
            row = find_product(inventory,nameid)
```

```
            if not row:
```

```
                print("Product not found")
```

```
                continue
```

```
            break
```

```
print("Product found")
```

```
print_row(row)
```

```
while True:
```

```
    units_purchased = int_input("\nUnits purchased (or type EXIT to cancel): ")
```

```
    if units_purchased == "EXIT":
```

```
        return None
```

```
    if(units_purchased > row[3]):
```

```
        print("Not enough stock")
```

```
    else:
```

```
        break
```

```
row[3] -= units_purchased
```

```
purchased_item = row.copy()
```

```
purchased_item[3] = units_purchased
```

```
print("\n**Added product to cart**\n")
```

```
return purchased_item
```

```
except Exception as e:
```

```
    print("Failed Operation {}".format(e))
```

```
def restock_product(inventory):
```

```
    """
```

description: Function that searches for product that the shop wants to restock and updates inventory table stock if that item is restocked

also check for invalid input and can change selling price of already available product

parameter : list that has inventory data

return type: list that contains details of restock

```
    """
```



```

try:
    boughtlist = []
    print("=="*24)
    supplier_name = input("Enter Supplier name:")
    while True:
        nameid = input("Enter Product name or id (or type EXIT to cancel): ")
        if nameid.upper() == "EXIT":
            return None, None
        row = find_product(inventory, nameid)
        if not row:
            print("Product not found")
            continue
        print("Product found")
        print_row(row)
        units_purchased = int_input("\nUnits purchased (or type EXIT to cancel): ")
        if units_purchased == "EXIT":
            return None, None
        purchased_price = float_input("\nPurchase Price :")
        while True:
            x = input("Update Selling price? (y / n)")
            x = x.lower()
            if(x == "n"):
                break
            elif(x == "y"):
                print("Old Price per-Unit :", row[4])
                new_price = int_input("ENter New selling price :")
                row[4] = new_price
                break

```

```

        else:
            print("\nInvalid Entry\n")
            row[3] += units_purchased
            purchased_item = row.copy()
            purchased_item[3] = units_purchased
            purchased_item[4] = purchased_price
            boughtlist.append(purchased_item)
            x = input("Add another product? (y / n)")
            x = x.lower()
            if(x == "n"):
                break
            elif(x == "y"):
                continue
            else:
                print("\nInvalid Entry\n")
            return supplier_name,boughtlist
    except Exception as e:
        print("Failed Operation {}".format(e))

```

```
def add_product(inventory):
```

```
    """
```

description: Function that adds product to the inventory and checks if the product already exists

also check for invalid input and has different inputs for purchase price and selling price

parameter : list that has inventory data

return type: list that contains details of new product

```
    """
```

```
    try:
```

```

boughtlist = []
print("=="*24)
supplier_name = input("Enter Supplier name (or type EXIT to cancel):")
while True:
    id = input("Enter product ID")
    if id.upper() == "EXIT":
        return None, None
    if find_product(inventory, id):
        print("Product ID already exists")
        continue
    name = input("Enter product name (or type EXIT to cancel):")
    brand = input("Enter brand name (or type EXIT to cancel):")
    stock = int_input("Enter Bought stock (or type EXIT to cancel):")
    if stock == "EXIT":
        return None, None
    boughtprice = float_input("Enter unit buying price :")
    sellprice = float_input("Enter unit Selling price :")
    while True:
        source = input("Enter Source of manufacture (Domestic/International) :")
        source = source.title()
        if (source != "Domestic" and source != "International"):
            print("Wrong source")
            continue
        break
    boughtlist.append([id, name.title(), brand, stock, boughtprice, source])
    inventory.append([id, name.title(), brand, stock, sellprice, source])
    x = input("Add another product? (y / n)")
    x = x.lower()

```

```

        if(x == "n"):
            break
        elif(x == "y"):
            continue
        else:
            print("\nInvalid Entry\n")
        return supplier_name,boughtlist
    except Exception as e:
        print("Failed Operation {}".format(e))

def canceled_restock(inventory,boughtlist):
    try:
        for item in boughtlist:
            product = find_product(inventory, item[0])
            if product:
                product[3] += item[3]
    except Exception as e:
        print("Failed to restock canceled product {}".format(e))

def find_product(inventory, nameid):
    """
    description: Helper function for product check
    parameter : list that has inventory data, product id
    return type: list of required product
    """
    try:
        for row in inventory:
            if row[0].lower() == nameid.lower() or row[1].lower() == nameid.lower():

```

```

        return row

    return None

except Exception as e:
    print("Failed Operation {}".format(e))

def int_input(string):
    """
    description: Helper function for invalid and exception check
    parameter : String prompt for input
    return type: int for valid input
    """
    while True:
        value = input(string)
        if(value.upper()=="EXIT"):
            return "EXIT"
        try:
            value = int(value)
            if(value<=0):
                print("Enter valid value")
            else:
                return value
        except ValueError:
            print("Please enter an valid value.")
        except Exception as e:
            print("Failed Operation {}".format(e))

def float_input(string):
    """

```

description: Helper function for invalid and exception check

parameter : String prompt for input

return type: float for valid input

"""

while True:

 value = input(string)

 try:

 value = float(value)

 if(value<=0):

 print("Enter valid value")

 else:

 return value

 except ValueError:

 print("Please enter an valid value")

 except Exception as e:

 print("Failed Operation {}".format(e))

def print_row(row):

 print("="*85)

 print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} | {:<14}".format("ID", "Shoe", "Brand",
"Available", "Price", "Source"))

 print("-"*85)

 print("{:<5} | {:<15} | {:<12} | {:<10} | {:<14} |
{:<14}".format(row[0],row[1],row[2],row[3],row[4],row[5]))

11.6 Write.py

def write_inventory(inventory):

 """

 description: Function that writes data into the file, this data is the updated inventory
 data

 parameter : list that contains the updated data for file

 return type: none, return without value to the main menu

"""

try:

 with open("Catalogue.txt", "w") as file:

 for row in inventory:

 line = "{},{},{},{},{},{}\n".format(row[0],row[1],row[2],row[3],row[4],row[5])

 file.write(line)

except Exception as e:

 print("Failed to update inventory" + str(e))